

Euclid's Elements — Book II

Proposition 5

Formal Reconstruction — Technical Documentation

Jurek Rózański
Military University of Technology, Warsaw, Poland
`jerzy.rozanski@wat.edu.pl`

CGJTEAMLAB

If a straight line is bisected and also cut at another point, the rectangle contained by the two unequal parts together with the square on the segment between the two cut points equals the square on the half.

1 Introduction

Proposition II.5 is the first Book II theorem in the present reconstruction to combine a *midpoint condition* with the rectangle/scissors calculus. Let C bisect AB , and let D be a second cut point. In the ordering chosen by the Lean theorem,

$$A - D - C - B,$$

so that C is the midpoint of AB and D lies in the left half. The theorem asserts, in geometric content language,

$$\text{Rect}(AD, DB) + \text{Sq}(CD) = \text{Sq}(CB).$$

The familiar modern algebraic shadow is obtained by writing the half-length as a and the displacement of the unequal cut from the midpoint as x :

$$(a - x)(a + x) + x^2 = a^2.$$

This formula is only an interpretive mnemonic. The Lean proof does not define multiplication of segment lengths and does not evaluate numerical area. Its objects are concrete rectangle and square representatives, while equality is expressed by exact scissors equivalence.

The current source begins with

```
import CGJteamLab.Proposition2_4
```

but this import fact should not be confused with the proof DAG. The proof does not invoke `euclid_proposition_2_4`. It directly reuses the binary rectangle theorem from II.1 and Proposition II.3, together with the reusable rectangle representation API. Thus II.5 provides an especially useful example of the difference between module dependency and mathematical proof dependency.

2 Euclid's Statement and Classical Configuration

2.1 The statement

In the notation used here, Euclid's proposition may be stated as follows. A straight line AB is divided equally at C and unequally at D . Then the rectangle contained by AD and DB , together with the square on CD , is equal in content to the square on the half CB [?].

The phrase “rectangle contained by AD, DB ” names a plane figure whose adjacent sides have those magnitudes. It is not a product already formed in a segment field. Likewise, the addition in the displayed identity describes the content of two figures taken together.

2.2 The classical diagram

The principal figure below follows the standard Euclidean configuration with the unequal cut D chosen in the right half, so its baseline order is $A - C - D - B$. This is the mirror version of the Lean convention $A - D - C - B$; the content identity is unchanged. Euclid erects the square $CEFB$ on the half CB , joins BE , draws the parallel through D , then the horizontal through its intersection H with the diagonal, and finally the parallel through A [?].

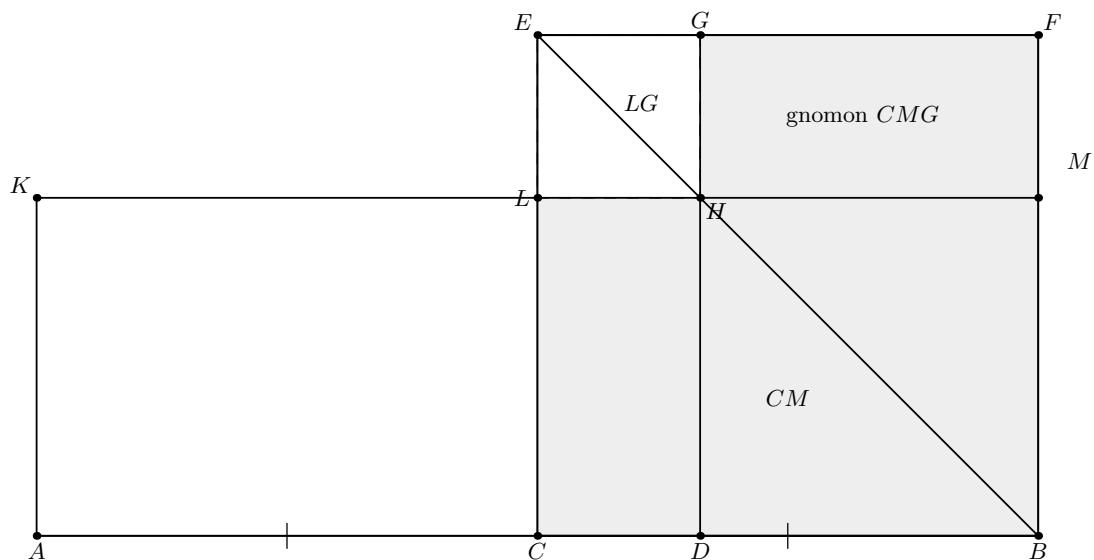


Figure 1: The classical mechanism of Euclid II.5. The square $CEFB$ is built on the half CB . The point D is the unequal cut, H lies on diagonal BE , and LG is the small square on CD . The remaining part of $CEFB$ is the gnomon used in Euclid’s content argument. The formal Lean proof does not introduce a primitive gnomon object.

3 Classical Proof

The classical proof uses the gnomon structure of Figure 1. The diagonal BE and the parallels create complementary parallelograms. By I.43 the complements CH and HF are equal. Adding the same rectangle DM to the relevant figures yields an equality in which the rectangle CM can be replaced by AL : these are equal parallelograms on equal bases AC and CB and in the same parallels, using I.36 and the midpoint condition $AC = CB$ [?].

The construction also gives $DH = DB$ by the square-of-a-sum geometry already available at II.4 (in Euclid’s classical dependency chain). Consequently the figure AH is the rectangle contained by the unequal parts AD and DB . Euclid identifies that rectangle with the gnomon obtained by removing the small corner square LG from the square $CEFB$. Finally LG is the square on the segment CD , and adjoining it to the gnomon recovers the entire square on CB .

Thus the classical local DAG is schematically

$$\begin{array}{c}
\text{I.43: equal complements} \quad + \quad \text{I.36: equal parallelograms on equal bases} \\
\text{together with the II.4 corollary} \\
\Downarrow \\
\text{rectangle on } AD, DB = \text{gnomon in the square on } CB \\
\Downarrow \\
\text{add the square on } CD \\
\Downarrow \\
\text{Rect}(AD, DB) + \text{Sq}(CD) = \text{Sq}(CB).
\end{array}$$

This is *not* the Lean DAG. In particular, the current formal proof does not reconstruct the classical gnomon, does not call I.43 locally, and does not call II.4 despite importing the II.4 module.

4 The Geometric Identity in the Current Formalization

The public Lean theorem chooses the strict order

$$A - D - C - B$$

through two hypotheses:

```

(hMidC : HilbertIsMidpoint Geo C A B)
(hADC : Geo.Between A D C)

```

The midpoint hypothesis supplies $A - C - B$ and $AC \cong CB$. From $A - D - C$ and $A - C - B$, a private order helper derives $D - C - B$ and $A - D - B$. Thus the full four-point order is a theorem of the hypotheses, not an additional assumption.

At the exact-scissors level the target is

$$\text{Rect}(DB, AD) + \text{Sq}(DC) \simeq_{\text{sc}} \text{Sq}(CB).$$

The reversed order DB, AD rather than AD, DB is immaterial at the geometric rectangle level because side-order exchange is itself proved by rectangle rotation/transport. It is not treated as primitive commutativity of a segment product.

A notable semantic point is that no gnomon type occurs in the source. The classical gnomon is represented extensionally by formal sums of rectangle and square scissors terms. The proof then rearranges and composes those sums by `HilbertScissorsEq.add`, symmetry, transitivity, and the existing rectangle transport lemmas.

5 Reusable Book II Infrastructure

5.1 The binary rectangle split from II.1

The theorem `euclid_proposition_2_1_two` is used twice. First it splits DB at C , with AD held as the other containing side:

$$\text{Rect}(DB, AD) \simeq_{\text{sc}} \text{Rect}(DC, AD) + \text{Rect}(CB, AD).$$

Later the same binary theorem, now applied to AC split at D , assembles

$$\text{Rect}(CB, AD) + \text{Rect}(CB, DC) \simeq_{\text{sc}} \text{Rect}(CB, AC),$$

after geometric side-order transports.

5.2 Proposition II.3

Proposition II.3 supplies the first quadratic-looking decomposition:

$$\text{Rect}(AC, DC) \simeq_{\text{sc}} \text{Rect}(AD, DC) + \text{Sq}(DC).$$

The current source calls `euclid_proposition_2_3` directly. The midpoint congruence $AC \cong CB$ is then used to transport the left-hand rectangle to a representative contained by CB, DC .

5.3 Rectangle representation and transport

The proof relies on the established Book II rectangle API rather than numerical products. In particular it uses

- `rectangle_transport_scissorsEq` to change containing segments along segment congruences;
- `rectangle_contained_by_comm` and `rectangle_contained_by_swap` to exchange the two adjacent side roles geometrically;
- `rectangle_contained_by_unique` to compare arbitrary representatives of the same contained-by data;
- `square_is_rectangle_contained_by_side` to view a square as a rectangle contained by one side twice.

6 Proof Architecture of the Reconstruction

The production file preserves the private helper names created during the incremental proof search. The names contain `testXX`, but all these lemmas are `private`; they are implementation history, not public API. Their mathematical roles are stable and form the following chain.

6.1 Step 1: recover the full order and midpoint congruence

`proposition2_5_test01_order` derives

$$D - C - B, \quad A - D - B, \quad AC \cong CB, \quad CA \cong CB.$$

This is the only place where the four-point order is consolidated.

6.2 Step 2: apply II.3 on the left half

`proposition2_5_test02_left_decomposition` proves

$$\text{Rect}(AC, DC) \simeq_{\text{sc}} \text{Rect}(AD, DC) + \text{Sq}(DC).$$

6.3 Step 3: transport the midpoint side

`proposition2_5_test03_midpoint_rectangle_transport` uses $AC \cong CB$ to prove

$$\text{Rect}(AC, DC) \simeq_{\text{sc}} \text{Rect}(CB, DC).$$

By symmetry and transitivity, `proposition2_5_test04_first_assembled_identity` obtains

$$\text{Rect}(CB, DC) \simeq_{\text{sc}} \text{Rect}(AD, DC) + \text{Sq}(DC).$$

6.4 Step 4: split the unequal part DB

`proposition2_5_test05_db_decomposition` applies the binary II.1 theorem to $D - C - B$:

$$\text{Rect}(DB, AD) \simeq_{\text{sc}} \text{Rect}(DC, AD) + \text{Rect}(CB, AD).$$

The next two helpers use geometric commutativity to normalize the first term:

$$\text{Rect}(DB, AD) \simeq_{\text{sc}} \text{Rect}(AD, DC) + \text{Rect}(CB, AD).$$

6.5 Step 5: the scissors bridge

The private lemma `proposition2_5_test08_scissors_bridge` is an abstract content calculation. Given

$$P \simeq X + Y, \quad Z \simeq X + S,$$

it proves

$$P + S \simeq Y + Z.$$

Crucially, this is *not* a cancellation principle. It is obtained by adding exact scissors equalities, reassociating and commuting the formal multiset sum, and using transitivity. Applying it to the two preceding identities gives

$$\text{Rect}(DB, AD) + \text{Sq}(DC) \simeq_{\text{sc}} \text{Rect}(CB, AD) + \text{Rect}(CB, DC).$$

6.6 Step 6: assemble the right side

`proposition2_5_test09_assemble_right_side` rotates the two right-hand rectangle representatives, applies the binary II.1 split to $A - D - C$, and rotates the result back. It proves

$$\text{Rect}(CB, AD) + \text{Rect}(CB, DC) \simeq_{\text{sc}} \text{Rect}(CB, AC).$$

6.7 Step 7: recognize the half-square

Finally `proposition2_5_test10_half_rectangle_is_square` uses $AC \cong CB$ and representative uniqueness to identify

$$\text{Rect}(CB, AC) \simeq_{\text{sc}} \text{Sq}(CB).$$

This is where the midpoint hypothesis enters a second time: first it controlled order, and now it turns the final rectangle into the square on the half.

7 Lean Formalization

7.1 The public theorem signature

The following is the current compiling public signature. It is intentionally large because the theorem receives the concrete rectangle-cut configurations and representatives required by the earlier Book II APIs rather than hiding them behind choice constructions.

```
theorem euclid_proposition_2_5
  [HilbertIncidence Geo]
  [HilbertEuclideanPlane Geo]

  (A B C D : Geo.Point)
  (hMidC : HilbertIsMidpoint Geo C A B)
  (hADC : Geo.Between A D C)

  -----
  -- Data used by test 04 inside test 08.
  -----

  (E4 H4 L4 X4 : Geo.Point)
  (hRect4 : IsRectangle Geo A C H4 E4)
  (hCH_DC : Geo.Congruent C H4 D C)
  (hELE4 : Geo.Between E4 L4 H4)
  (hEXC4 : Geo.Between E4 X4 C)
  (hDXL4 : Geo.Between D X4 L4)
  (hLeftPar4 : IsParallelogram Geo L4 E4 A D)
  (hRightPar4 : IsParallelogram Geo C H4 L4 D)

  -- Square on DC.
  (F G : Geo.Point)
  (hSquareDC : IsSquare Geo D C F G)

  -- Rect(AC,DC).
  (R0 R1 R2 R3 : Geo.Point)
  (hAC_DC :
    IsRectangleContainedBy Geo
      R0 R1 R2 R3 A C D C)

  -- Common Rect(AD,DC).
  (T0 T1 T2 T3 : Geo.Point)
  (hAD_DC :
    IsRectangleContainedBy Geo
      T0 T1 T2 T3 A D D C)

  -- Rect(CB,DC).
```

```

(Z0 Z1 Z2 Z3 : Geo.Point)
(hCB_DC :
  IsRectangleContainedBy Geo
    Z0 Z1 Z2 Z3 C B D C)

-----

-- Data used by test 07 inside test 08.
-----

(E7 H7 L7 X7 : Geo.Point)
(hRect7 : IsRectangle Geo D B H7 E7)
(hBH_AD : Geo.Congruent B H7 A D)
(hELE7 : Geo.Between E7 L7 H7)
(hEXB7 : Geo.Between E7 X7 B)
(hCXL7 : Geo.Between C X7 L7)
(hLeftPar7 : IsParallelogram Geo L7 E7 D C)
(hRightPar7 : IsParallelogram Geo B H7 L7 C)

-- Rect(DB,AD), the unequal-parts rectangle.
(P0 P1 P2 P3 : Geo.Point)
(hDB_AD :
  IsRectangleContainedBy Geo
    P0 P1 P2 P3 D B A D)

-- Rect(DC,AD), before commutativity normalization.
(Q0 Q1 Q2 Q3 : Geo.Point)
(hDC_AD :
  IsRectangleContainedBy Geo
    Q0 Q1 Q2 Q3 D C A D)

-- Rect(CB,AD).
(Y0 Y1 Y2 Y3 : Geo.Point)
(hCB_AD :
  IsRectangleContainedBy Geo
    Y0 Y1 Y2 Y3 C B A D)

-----

-- Data for test 09:
-- assemble Rect(CB,AD) + Rect(CB,DC) into Rect(CB,AC).
-----

(E9 H9 L9 X9 : Geo.Point)
(hRect9 : IsRectangle Geo A C H9 E9)
(hCH_CB9 : Geo.Congruent C H9 C B)
(hELE9 : Geo.Between E9 L9 H9)
(hEXC9 : Geo.Between E9 X9 C)
(hDXL9 : Geo.Between D X9 L9)
(hLeftPar9 : IsParallelogram Geo L9 E9 A D)
(hRightPar9 : IsParallelogram Geo C H9 L9 D)

-- Rect(CB,AC).
(W0 W1 W2 W3 : Geo.Point)
(hCB_AC :
  IsRectangleContainedBy Geo
    W0 W1 W2 W3 C B A C)

-----

-- Data for test 10: square on CB.

```

```

-----
(S0 S1 S2 S3 : Geo.Point)
(hSquareCB : IsSquare Geo S0 S1 S2 S3)
(hS01_CB : Geo.Congruent S0 S1 C B) :

HilbertScissorsEq Geo
  (hilbertParallelogramTerm Geo P0 P1 P2 P3 +
   hilbertParallelogramTerm Geo D C F G)
  (hilbertParallelogramTerm Geo S0 S1 S2 S3) := by

```

Several layers of data are visible in the signature.

1. A, B, C, D and the midpoint/betweenness hypotheses encode the one-dimensional configuration.
2. The groups labelled in the source as data for tests 04, 07, and 09 are concrete rectangle-cut witnesses needed by II.3 or II.1.
3. F, G give a square on DC .
4. $P_0P_1P_2P_3$ represents the rectangle contained by DB, AD .
5. $W_0W_1W_2W_3$ represents the intermediate rectangle contained by CB, AC .
6. $S_0S_1S_2S_3$ is an arbitrary square representative whose side is congruent to CB .

Thus the theorem proves representative-independent content equality for supplied geometric witnesses; it does not claim that all of those witnesses are constructed internally from four points alone.

7.2 The final three-stage assembly

After all private helpers have been established, the public theorem is short in conceptual structure. The current source builds exactly three equations:

1. h08: unequal-parts rectangle plus the DC square equals two rectangles with common side CB ;
2. h09: those two rectangles assemble into the rectangle $\text{Rect}(CB, AC)$;
3. h10: midpoint congruence turns that rectangle into the square on CB .

The final transitivity block is:

```

have h08 :
  HilbertScissorsEq Geo
    (hilbertParallelogramTerm Geo P0 P1 P2 P3 +
     hilbertParallelogramTerm Geo D C F G)
    (hilbertParallelogramTerm Geo Y0 Y1 Y2 Y3 +
     hilbertParallelogramTerm Geo Z0 Z1 Z2 Z3) :=
proposition2_5_test08_core_identity
Geo
A B C D

```



```

hMidC hADC
E4 H4 L4 X4
hRect4 hCH_DC hELE4 hEXC4 hDXL4
hLeftPar4 hRightPar4
F G hSquareDC
R0 R1 R2 R3
hAC_DC
T0 T1 T2 T3
hAD_DC
Z0 Z1 Z2 Z3
hCB_DC
E7 H7 L7 X7
hRect7 hBH_AD hELE7 hEXB7 hCXL7
hLeftPar7 hRightPar7
P0 P1 P2 P3
hDB_AD
Q0 Q1 Q2 Q3
hDC_AD
Y0 Y1 Y2 Y3
hCB_AD

-----

-- test09:
-- Rect(CB,AD) + Rect(CB,DC) = Rect(CB,AC).
-----

have h09 :
  HilbertScissorsEq Geo
    (hilbertParallelogramTerm Geo Y0 Y1 Y2 Y3 +
     hilbertParallelogramTerm Geo Z0 Z1 Z2 Z3)
    (hilbertParallelogramTerm Geo W0 W1 W2 W3) :=
proposition2_5_test09_assemble_right_side
  Geo
  A B C D
  hADC
  E9 H9 L9 X9
  hRect9 hCH_CB9 hELE9 hEXC9 hDXL9
  hLeftPar9 hRightPar9
  Y0 Y1 Y2 Y3
  hCB_AD
  Z0 Z1 Z2 Z3
  hCB_DC
  W0 W1 W2 W3
  hCB_AC

-----

-- test10:
-- Rect(CB,AC) = Square(CB).
-----

have h10 :
  HilbertScissorsEq Geo
    (hilbertParallelogramTerm Geo W0 W1 W2 W3)
    (hilbertParallelogramTerm Geo S0 S1 S2 S3) :=
proposition2_5_test10_half_rectangle_is_square
  Geo
  A B C D
  hMidC hADC

```

```

W0 W1 W2 W3
hCB_AC
S0 S1 S2 S3
hSquareCB
hS01_CB

-----

-- Full II.5.
-----

exact
  HilbertScissorsEq.trans
    (Geo := Geo)
  h08
  (HilbertScissorsEq.trans
    (Geo := Geo)
  h09

```

The formal result is therefore `exact HilbertScissorsEq`, not the weaker relation `HilbertScissorsEquicomplementable`, and not equality of real numbers.

8 Relationship with Earlier Book II Propositions

8.1 Proposition II.1

II.1 is the basic additive engine of the proof. Its binary theorem is called directly in the decomposition of DB and again in the assembly of AC . Thus II.5 genuinely reuses the rectangle split calculus rather than reproving the corresponding dissections from Book I geometry.

8.2 Proposition II.3

II.3 is also called directly. It supplies the identity

$$\text{Rect}(AC, DC) = \text{Rect}(AD, DC) + \text{Sq}(DC),$$

at the exact scissors level. This is the step that inserts the square on the difference between the two section points.

8.3 Proposition II.4

The source file imports `CGJteamLab.Proposition2_4`, so II.4 is a direct *module* dependency. Nevertheless the theorem `euclid_proposition_2_4` is not invoked in the proof term. This sharply contrasts with Euclid’s classical proof, where a corollary of II.4 participates in the identification of the relevant rectangle and small square.

This distinction should be recorded explicitly:

module chain: II.4 module $\rightarrow$ II.5 module,
 Lean theorem calls: II.1 binary API + II.3 + rectangle API.

9 Relationship with Book I Infrastructure

The Book I geometry is present below the reusable rectangle layer. Rectangles, squares, right angles, parallel sides, parallelograms, and finite scissors terms all ultimately depend on the established Hilbert/Book I development. II.5 does not locally replay Euclid’s diagonal-complement argument from I.43. Instead, the effect of those earlier geometric facts has already been packaged into `HilbertRectangle`, the scissors API, and the earlier Book II theorems.

This is an architectural gain: a proposition whose classical proof is diagram specific becomes a composition of stable content-level interfaces.

10 Formalization Notes and Hidden Geometric Data

10.1 Order orientation

The classical diagram commonly places the unequal cut in the right half $A - C - D - B$, whereas the Lean theorem fixes $A - D - C - B$. These are mirror cases. The source makes the chosen orientation explicit through `hADC` and then derives $D - C - B$ from the midpoint betweenness. No appeal to a picture is allowed to supply this ordering silently.

10.2 Midpoint as geometry, not arithmetic

`HilbertIsMidpoint` contributes both betweenness and segment congruence. The proof uses those components separately. Betweenness creates the required subsegment orders; congruence transports rectangle sides and eventually recognizes the square on the half. There is no variable standing for a numerical half-length.

10.3 Representative independence

The objects named “the rectangle contained by” particular segments are concrete quadrilateral representatives. Different helpers may receive different representatives of the same pair of side magnitudes. `rectangle_transport_scissorsEq`, `rectangle_contained_by_unique`, and the side-order transport theorems are therefore mathematically substantive parts of the formal statement, not mere syntactic rewrites.

10.4 No gnomon primitive

Euclid’s proof naturally names a gnomon. The present library does not define a `GnomonTerm` or another primitive gnomon object for II.5. No such new abstraction is needed here: the required content is expressed as a formal sum of scissors terms. The private scissors bridge then performs the exact content rearrangement that the classical gnomon packages visually.

10.5 No cancellation axiom

At an intermediate stage the two identities contain the common rectangle $\text{Rect}(AD, DC)$. It might be tempting to introduce a cancellation principle for content. The actual source deliberately

avoids this. The scissors bridge combines equalities additively and rearranges formal sums, so no new positivity or cancellation theorem is required.

11 Axiomatic and Semantic Status

The public theorem assumes

```
[HilbertIncidence Geo]
[HilbertEuclideanPlane Geo]
```

and the production file introduces no new proposition-specific `axiom`, `sorry`, or `admit`. The user-verified module build is clean. The reported transitive axiom audit is

```
#print axioms Geometry.euclid_proposition_2_5

'Geometry.euclid_proposition_2_5' depends on axioms: [propext,
Classical.choice,
Geometry.bookZero_nullSegment1,
Quot.sound]
```

Accordingly II.5 should not be described as “axiom-free.” The precise claim is that II.5 introduces no new proposition-specific axiom and no `sorry`; its transitive dependencies are the four declarations shown above, and the list contains no `sorryAx`.

12 Dependency Summary

The actual Lean dependency chain is best displayed as

$$\begin{array}{c}
 \text{Hilbert incidence} + \text{Euclidean plane layer} \\
 \Downarrow \\
 \text{Book I square, rectangle, parallelogram, and scissors infrastructure} \\
 \Downarrow \\
 \text{HilbertRectangle representative/transport API} \\
 \Downarrow \\
 \text{II.1 binary split} + \text{II.3 rectangle-plus-square theorem} \\
 \Downarrow \\
 \text{private II.5 order, transport, bridge, and assembly helpers} \\
 \Downarrow \\
 \text{public theorem euclid_proposition_2_5.}
 \end{array}$$

The classical local chain is different:

$$\text{I.43} + \text{I.36} + \text{II.4 corollary} \longrightarrow \text{gnomon identification} \longrightarrow \text{II.5.}$$

The formal proof is therefore not a line-by-line translation of the classical one. It reconstructs the same geometric content through a different, reusable DAG.

New geometric axiom in II.5:	no
New content axiom in II.5:	no
New proposition-local helper theorem:	yes, private helpers
New reusable public helper theorem:	no
New <code>HilbertRectangle</code> theorem:	no
New Interface / Book Zero / Scissors theorem:	no
Directly reuses earlier Book II theorem:	yes, II.1 and II.3
Imports II.4 module:	yes
Calls <code>euclid_proposition_2_4</code> :	no
Uses exact <code>HilbertScissorsEq</code> :	yes
Uses general equicomplementability:	no
Uses numerical area:	no
Uses segment multiplication:	no
Public theorem compiles:	yes
Module build:	clean (user verified)
Proposition-local <code>sorry/admit</code> :	no
Transitive <code>sorryAx</code> :	absent in reported audit

13 Conclusion

Proposition II.5 is an important architectural checkpoint for Book II. The classical proof organizes the equality around a gnomon inside the square on the half. The formal proof shows that a separate gnomon primitive is not needed to recover the theorem: the same content can be assembled from the binary II.1 rectangle split, II.3, midpoint congruence, exact rectangle transport, and the additive structure of `HilbertScissorsEq`.

The result also clarifies the role of the modern quadratic identity. It is a useful shadow for recognizing the proposition, but the verified argument stays synthetic throughout. Geometry supplies the order and congruences; representation supplies rectangles and squares; scissors equality supplies the content calculus. Numerical area and segment multiplication remain outside the proof.